

# 웹뷰 기반 실시간 서비스를 설계하고 팀 개발 기준을 세우는 Frontend Engineer



정다훈

Frontend Engineer · 총 5년 9개월

|           |                        |       |                    |
|-----------|------------------------|-------|--------------------|
| 휴대폰       | 010-4346-0429          | Email | dahun428@naver.com |
| 주소        | 경기 구리시 인창동             | 병역    | [군필] 육군 병장         |
| Portfolio | github.com/dahun428-fx |       |                    |

## 요약

React·Next.js·TypeScript·TanStack Query를 기반으로 **웹뷰 환경의 실시간 서비스를 설계하고, 성능과 개발 생산성을 측정 가능한 수치로 개선해 온 프론트엔드 개발자**입니다. 5년 9개월 동안 AI 서비스 제품화, 레거시의 Next.js 전환, 웹뷰 운영, 팀 개발 기준 수립을 동시에 맡으며 한 서비스의 개발·운영·고도화 생애주기를 여러 번 직접 돌려 밀도 있게 경험을 쌓았습니다. AI 건강검진 챗봇에서 SSE 기반 실시간 응답 스트리밍과 오류 가드레일을 실서비스에 도입해 답변 출력 시간을 10초에서 4초로, 초기 진입 시간을 30초에서 6초로 단축했고, 웹뷰 환경에서 React 화면을 운영하며 iOS·Android 호환성 검증 기준을 세웠습니다.

페이지 특성에 맞춰 SSR·CSR 렌더링 전략을 분리하고, Next.js 전환으로 페이지 접근 시간을 8초에서 2초로 줄였습니다. Playwright E2E 자동화 600여 건·Jest 단위 테스트·Storybook 기반 컴포넌트 검증으로 서비스 안정성을 관리하고, FE AX SOP 문서화·개발 세미나 12회·기술 문서 48건으로 팀이 같은 기준으로 코드 품질과 AI 도구 활용을 관리하도록 정착시켰습니다.

Cursor·Claude 등 LLM 기반 AI 도구를 실무에 적용하고 팀 도입 기준을 문서로 정립한 경험이 있으며, Jira·Confluence·Figma·Slack로 기획·AI·백엔드·운영 조직과 API 응답 정책·렌더링 전략·일정을 직접 조율해 왔습니다.

## 핵심 성과

### 1. 웹뷰 기반 AI 서비스 실시간 응답 처리 및 진입 성능 개선

SSE 기반 실시간 응답 처리와 답변 중지·재시도·오류 상태 흐름을 설계해 웹뷰 환경 AI 건강검진 챗봇에 실서비스로 도입. Markdown 전용 렌더링 계층·XSS 필터링·오류 가드레일 적용, React Query 캐싱·코드 스플리팅·HTTP/2 전환으로 AI 답변 출력 10초 → 4초, 초기 진입 30초 → 6초, Lighthouse 91점

### 2. SSR·CSR 렌더링 전략 분리 및 Next.js 웹 성능 개선

PHP·jQuery 기반 글로벌 B2B 커머스를 Next.js·React·TypeScript 구조로 전환, 페이지 특성별 렌더링 전략 분리. Lazy Loading·비동기 API·리소스 최적화로 페이지 접근 8초 → 2초

### 3. Playwright E2E·Jest 단위 테스트 기반 서비스 안정성 관리

600여 건 E2E 회귀 시나리오·Jest 단위 테스트 자동화로 반복 QA 3시간 → 1시간, 테스트 커버리지 98%. Jenkins CI/CD 정비로 배포 리드타임 10분 → 2분

### 4. LLM 기반 AI 도구 실무 적용 및 팀 개발 기준 수립·멘토링

Cursor·Claude 등 AI 도구를 실무에 적용하고 FE AX SOP로 팀 도입 기준 정립. 개발 세미나 12회·기술 문서 48건, 반복 컴포넌트 개발·검증 90분 → 15분(약 83%)

### 5. 관측성 도구 기반 지표 분석 및 서비스 개선

Datadog·GA4·PostHog·Lighthouse·Adobe Analytics로 성능·오류·사용자 행동 지표 수집. 운영 데이터 확인 3단계 → 1단계 대시보드 전환, HTTP/2로 네트워크 비용 1,200KB → 900KB(약 25%)

### 6. 크로스팀 협업 및 서비스 고도화 주도

기획·AI·백엔드·운영 조직과 API 응답 형식·스트리밍 방식·오류 정책 직접 조율. WBS·ETA 기반 다중 서비스 동시 진행, 고객사별 CI·메뉴·기능 노출 변경 1주 내 대응 가능한 운영 구조 구축

## 총 5년 9개월

2025.07 ~ 재직중

## 대웅제약 AI추진팀

주요직무: Frontend Engineer / 프론트엔드 아키텍처 설계·개발 / AI 서비스 제품화 / 팀 개발 기준 수립

React·TypeScript 기반 프론트엔드 아키텍처를 설계하고 AI 서비스 제품화와 B2B 플랫폼 내재화를 주도했습니다. SSE 기반 실시간 응답 처리와 웹뷰 운영, Playwright·Jest·Storybook 기반 테스트 자동화, LLM 도구 실무 적용과 FE AX SOP 문서화로 팀 개발 기준을 정착시켰습니다.

- SSE 기반 실시간 LLM 응답 스트리밍·오류 가드레일 실서비스 도입 (실시간 스트리밍)
- 웹뷰 환경 React 화면 운영 및 iOS·Android 호환성 검증 기준 수립 (웹뷰 기반 개발)
- LLM 답변 출력 10초 → 4초, 초기 진입 30초 → 6초, Lighthouse 91점
- Playwright 600여 건 E2E·Jest 단위 테스트 자동화, 반복 QA 3시간 → 1시간 (단위·e2e 테스트)
- Jenkins CI/CD 빌드·배포 파이프라인 정비, 배포 리드타임 10분 → 2분
- Cursor·Claude 등 LLM 도구 실무 적용, FE AX SOP 문서화·개발 세미나 12회·기술 문서 48건 (AI 도구 팀 도입)
- 기획·AI·백엔드·운영 조직과 API 응답 정책·기능 노출 조건 조율, WBS·ETA 기반 일정 관리 (Jira·Confluence·Figma·Slack)

2020.10 ~ 2025.06

## 내담씨앤씨 SI사업부

주요직무: 웹 프론트엔드 개발자 / PC·모바일 웹 개발 / 레거시 전환

글로벌 B2B 커머스·데이터 플랫폼·모바일 앱 프로젝트에서 프론트엔드 아키텍처 개선과 성능 최적화를 수행했습니다. PHP·jQuery 레거시를 React·Next.js·TypeScript 구조로 단계적으로 전환하고, Vue 3·React Native로 PC 웹과 모바일 서비스를 함께 개발했습니다.

- PHP·jQuery 기반 글로벌 B2B 커머스를 React·Next.js·TypeScript 구조로 단계적 전환 (레거시 점진 전환)
- 페이지 특성별 SSR·CSR 렌더링 전략 분리, Lazy Loading·비동기 API·리소스 최적화로 페이지 접근 8초 → 2초 (SSR·CSR 대응)
- 상품 비교·멀티 다운로드 등 복잡한 기능을 공통 컴포넌트·Custom Hook으로 모듈화
- Vue 3·ECharts·RealGrid 기반 대용량 데이터 대시보드 Lazy Rendering 적용, 렌더링 2.5초 → 1초대
- React Native 기반 iOS·Android 앱 기능 고도화 및 플랫폼별 이슈 대응, 검색 응답 5초 → 1초
- Vercel 배포 자동화, Lighthouse·GA·Adobe Analytics·Datadog 기반 성능·SEO·오류 추적 체계 운영 (관측성)

## Frontend

React, Next.js, Vue 3, React Native, TypeScript, JavaScript(ES6+), Redux, Recoil, TanStack Query, RxJS, Tailwind CSS, SCSS, i18next

- React·Next.js 기반 **웹뷰 환경 실시간 서비스** 설계·개발·운영 (웹뷰 기반 FE)
- Next.js 기반 SSR·CSR 렌더링 전략 분리, Lighthouse 병목 분석·최적화 (SSR·CSR)
- TanStack Query 기반 서버 상태 캐싱·동기화로 초기 진입·네트워크 병목 개선
- WebView 기반 모바일 환경에서 React 웹 화면 운영 및 호환성 검증

## 실시간 / 렌더링

SSE, Markdown Renderer, 렌더링 계층 분리, 코드 스플리팅, HTTP/2

- **SSE 단방향 스트리밍**으로 LLM 응답의 중지·재시도·오류 상태와 UI 동기화 처리 (실시간 스트리밍)
- Markdown 전용 렌더링 계층으로 표·목록·링크·차트를 React 컴포넌트로 변환
- 코드 스플리팅·이미지 최적화·HTTP/2 전환으로 초기 진입·네트워크 병목 개선

## 테스트 / 품질 / 배포

Playwright, Jest, Storybook, ESLint, Jenkins, GitLab CI/CD, Vercel

- Playwright 기반 **600여 건 E2E 회귀 시나리오**·Jest 단위 테스트 자동화, 반복 QA 3시간 → 1시간 (단위·e2e 테스트)
- Storybook 중심 컴포넌트 상태별 개발·검증 절차 수립
- Jenkins·GitLab CI/CD·Vercel 빌드·검증·배포 파이프라인 구축, 배포 리드타임 10분 → 2분

## AI 도구 / 개발 생산성

Cursor, Claude, FE AX SOP, 공통 컴포넌트, Config-Driven UI, Base-Theme

- Cursor·Claude 등 **LLM 기반 AI 도구를 실무 개발에 적용**하고 팀 도입 기준을 FE AX SOP로 문서화 (LLM 도구 실무·팀 도입)
- AI 생성 코드의 컨텍스트 관리·금지 패턴·보안 위험·검증 절차 문서화, 개발 세미나 12회·기술 문서 48건
- 고객사별 테마·기능 노출 차이를 Config-Driven UI·Base-Theme로 분리, 반복 컴포넌트 개발 90분 → 15분

## 관측성 / 데이터

Datadog, GA4, PostHog, Google Lighthouse, Adobe Analytics, ECharts, RealGrid

- Datadog·GA4·PostHog 기반 성능·오류·사용자 행동 지표 수집 및 개선 근거 확보 (관측성)
- Lighthouse 병목 분석 기반 렌더링·리소스 최적화
- 대용량 테이블·시계열 데이터 시각화 및 Lazy Rendering 최적화

## Backend / API

Node.js(Express), Java, Spring Boot, REST API, MySQL, MyBatis, jQuery, Thymeleaf

- 비즈36.5에서 여러 백엔드 응답을 React 프론트 요구에 맞게 집계·프록시하는 BFF REST API를 Node.js(Express)로 신규 구축
- MySQL 데이터 모델·Spring Boot 로직·REST API 설계 및 Web·Admin End-to-End 개발

## 협업

Jira, Confluence, Figma, Slack

- Jira·Confluence 기반 이슈·일정 관리 및 기술 문서·FE AX SOP 공유 (협업 도구)
- Figma 기반 디자인 핸드오프·검수, Slack 기반 기획·디자인·BE·운영 실시간 커뮤니케이션

## 학력

2010.03 ~ 2017.02      **성공회대학교 일어일본학과 졸업**  
복수전공: 경영학과  
학점: 3.8 / 4.5

2010      **여의도고등학교 졸업**

## 교육

**2020.03 ~ 2020.09**    웹 콘텐츠 개발을 위한 응용SW엔지니어링 / 중앙에이치티에이주

Java 기반 웹 애플리케이션의 백엔드와 프론트엔드 개발 과정 이수

### 습득 기술:

HTML5, JavaScript, jQuery, Vue.js, CSS, JSP, Java, Spring Framework, AWS, RESTful API, OracleDB, MariaDB, Git/GitHub

## 자격증

**정보처리기사**      한국산업인력공단

## 어학

**영어**      TOEIC 825점  
취득일: 2024.05

**일본어**      JLPT 1급  
취득일: 2018.08

## 《 대응계약 | 웹뷰 기반 AI 실시간 서비스 도입·성능 개선 》

## SSE 기반 실시간 LLM 응답과 웹뷰 운영, Config-Driven UI 기반 모듈화 구조 설계

2025.07 ~ 2025.10

**책임 범위** 프론트엔드 아키텍처 설계·개발, 기획·AI·백엔드 조직과 API 응답 정책 조율, 실시간 응답 처리, 웹뷰 운영, 공통 컴포넌트 기반 서비스 확장 구조 구축

**[문제]**                    목업 수준의 AI 건강검진 챗봇을 실사용 가능한 서비스로 고도화해야 했습니다. 초기 PoC는 답변 생성 후 일괄 출력하는 구조라 체감 대기 시간이 길었고, Markdown·표·링크·차트 등 비정형 실시간 응답을 안정적으로 렌더링하는 체계와 웹뷰 환경에서의 호환성 검증 기준이 없었습니다.

**[주요 실행]**

- AI 개발자·백엔드와 API 응답 형식·스트리밍 방식·오류 정책 조율 (크로스팀 협업)
- **SSE 기반 실시간 응답**, 답변 중지·재시도·오류 상태 흐름 설계 (실시간 스트리밍)
- Markdown 전용 렌더링 계층을 구현해 표·목록·링크·차트를 React 컴포넌트로 변환
- XSS 필터링과 404·500·LLM 오류 가드레일 적용으로 비정형 응답 출력 안정성 확보
- 웹뷰 환경 React 화면 운영 및 iOS·Android 호환성 검증 기준 수립 (웹뷰 기반 개발)
- React Query 캐싱·코드 스플리팅·이미지 최적화·HTTP/2 전환으로 초기 진입·네트워크 병목 개선

**[성과]**

- AI 답변 출력 시간을 **10초에서 4초로 단축**
- 초기 서비스 진입 시간을 **30초에서 6초로 단축**
- Lighthouse 91점 확보
- 3,493명 대상 PoC의 400건 발화 검증 기준 답변 화면 정상 출력률 100% 달성
- 신규 AI 챗봇 구축 기간을 **10주에서 2주로 단축** 가능한 공통 컴포넌트 구조 설계

**기술** React, TypeScript, TanStack Query, Recoil, SSE, Chart.js, Tailwind CSS, WebView

« 한국미스미 | 글로벌 B2B 커머스 Next.js 전환 및 SSR·CSR 렌더링 전략 분리 · 내담씨앤씨 SI 프로젝트 »

## PHP·jQuery 레거시 분석, Next.js·React 전환, 렌더링 전략·성능·다국어 ·모니터링 구조 개선

2022.06 ~ 2024.10

책임 범위 PHP·jQuery 레거시 분석, Next.js·React·TypeScript 단계적 전환, SSR·CSR 렌더링 전략 분리, 성능·SEO·다국어·배포 자동화·모니터링 구조 개선

**[문제]** PHP·jQuery 기반 글로벌 B2B 커머스 쇼핑몰은 화면 결합도가 높아 기능 확장과 유지보수가 어려웠습니다. 단일 렌더링 구조로 초기 페이지 접근 시간이 길고 SEO 최적화에 한계가 있었으며, 다국어 환경과 국가별 배포 요건을 지원하면서 단계적으로 현대화해야 했습니다.

**[주요 실행]**

- PHP·jQuery 레거시를 **Next.js·React·TypeScript 컴포넌트 구조**로 단계적 전환 (레거시 점진 전환)
- 페이지 특성별 **SSR·CSR 렌더링 전략 분리** (SSR·CSR 대응)
- 상품 비교·멀티 다운로드 등 복잡한 기능을 공통 컴포넌트·Custom Hook으로 구조화
- Lighthouse 병목 분석 후 Lazy Loading·비동기 API·폰트·리소스 최적화 적용
- i18next 기반 글로벌 다국어 구조 구축
- Vercel 배포 자동화, Datadog·GA·Adobe Analytics 기반 운영 모니터링 체계 구축 (관측성)

**[성과]**

- 페이지 접근 시간을 **8초에서 2초로 단축**
- Next.js 전환 후 기존 PHP 서비스 대비 평균 로딩 속도 개선
- 화면 개발에서 아키텍처·성능·다국어·배포 자동화·모니터링까지 책임 범위 확장

기술 Next.js, React, TypeScript, JavaScript, Redux, RxJS, i18next, PHP, jQuery, Twig, Vercel, Datadog, Lighthouse, GA, Adobe Analytics

## 《 대응계약 | 테스트·빌드·배포 자동화 및 AI 도구 팀 개발 기준 수립 》

## Playwright E2E·Jest 단위 테스트 자동화, LLM 도구 실무 적용과 FE AX SOP 정착

2026년 상반기

적용 서비스 AI코치, 비즈36.5, 바이오에이지

책임 범위 E2E·단위 테스트 자동화, 빌드·배포 파이프라인 정비, 운영 지표 대시보드, LLM 도구 실무 적용, FE 개발 기준 문서화·세미나

**[문제]** 서비스 수와 공통 컴포넌트가 늘면서 수동 QA·반복 UI 개발·배포 전후 확인·운영 지표 조사가 병목이 됐습니다. Cursor·Claude 같은 LLM 도구로 생성한 코드의 품질·보안·일관성을 검증할 팀 공유 기준도 없어, 안정 운영을 위한 자동화 절차와 도입 기준이 필요했습니다.

**[주요 실행]**

- Playwright 기반 주요 사용자 흐름·E2E 회귀 시나리오와 Jest 단위 테스트 자동화 (단위·e2e 테스트)
- Storybook 중심 컴포넌트 상태별 개발·검증 절차 수립
- Jenkins CI/CD 빌드·검증·배포 파이프라인 정비
- GA4·PostHog 기반 운영 지표 대시보드 구성 (관측성 기반 개선)
- Cursor·Claude 등 LLM 도구를 실무에 적용하고, AI 생성 코드의 컨텍스트 관리·금지 패턴·보안 위험·검증 절차를 **FE AX SOP로 문서화** (LLM 도구 팀 도입)
- 개발 세미나 12회 운영·기술 문서 48건으로 팀 학습·적용·공유 체계 정착 (멘토링·기술 리딩)

**[성과]**

- Playwright 기반 **600여 건 E2E 회귀 시나리오** 자동화
- 반복 QA 시간을 **3시간에서 1시간으로 단축**
- 반복 컴포넌트 개발·검증 시간을 **90분에서 15분으로 단축** (약 83%)
- 운영 데이터 확인 절차를 3단계에서 1단계로 전환
- Jenkins CI/CD 배포 리드타임을 **10분에서 2분으로 단축**
- 테스트 커버리지 98% 수준 확보 (내부 측정 기준)

기술 Playwright, Jest, Storybook, ESLint, TypeScript, Jenkins, GitLab CI/CD, GA4, PostHog, Cursor, Claude



## « 대응계약 | B2B 임직원 건강 플랫폼 내재화 »

## 외주 서비스 내재화·고객사별 CI·메뉴·기능 노출 정책 공통화·WebView 앱 운영

2025.11 ~ 2026.02

**책임 범위** 서비스·데이터 구조 분석, jQuery·Thymeleaf→React 점진적 전환 설계, 사용자 권한·메뉴·브랜딩 정책 공통화, Jenkins CI/CD·WebView 운영

**[문제]** 외주 중심으로 운영되던 검진 예약 서비스를 내부 개발·운영 가능한 임직원 건강 플랫폼으로 전환해야 했습니다. jQuery·Thymeleaf 화면과 Spring Boot·MySQL 구조가 기능별로 결합되어 변경 영향 범위 파악이 어려웠고, 고객사별 CI·메뉴·기능 노출 정책이 일관되지 않아 운영 대응 속도를 높이기 어려웠습니다.

**[주요 실행]**

- jQuery·Thymeleaf와 React가 화면 단위로 공존하는 **점진적 전환 구조 설계** (레거시 점진 전환)
- 사용자 권한·메뉴·브랜딩·데이터 출력 정책을 공통 기준으로 정리해 고객사별 배포 대응 구조 확보
- 신규 건강관리 기능의 MySQL 데이터 모델·Spring Boot 로직·REST API·Web·Admin 화면을 End-to-End 개발
- Jenkins 빌드·검증·배포 파이프라인과 WebView 호환성 검증 기준 수립 (웹뷰 기반 운영)

**[성과]**

- 9주 내 **108개 페이지·82개 화면**을 임직원 건강 플랫폼으로 전환
- 신규 건강관리 기능 3건을 데이터 모델부터 화면까지 End-to-End 개발
- 고객사별 CI·메뉴·기능 노출 변경을 **1주 내 대응 가능한 운영 구조** 확보

**기술** React, TypeScript, jQuery, Thymeleaf, Java, Spring Boot, REST API, MySQL, Tailwind CSS, Jenkins CI/CD, WebView, iOS, Android

« 삼성물산 | 데이터 플랫폼·모바일 애플리케이션 · 내담씨앤씨 SI 프로젝트 »»

## Vue 3 기반 대용량 데이터 대시보드 성능 최적화 및 React Native 모바일 서비스 고도화

2024.10 ~ 2025.04

책임 범위 대용량 데이터 시각화 성능 개선, Spring REST API-데이터 연동, React Native iOS·Android 개발·배포

**[문제]** Web 대시보드는 대용량 테이블·시계열 데이터를 빠르게 제공해야 했고, 모바일 앱은 반복 API 호출과 플랫폼별 동작 차이로 사용자 대기 시간과 운영 부담이 발생했습니다.

**[주요 실행]**

- Vue 3·ECharts·RealGrid 기반 대용량 데이터 시각화 화면 개발
- 테이블·차트 렌더링 생명주기를 분리하고 **Lazy Rendering**을 적용해 초기 처리량 최적화 (성능 개선)
- Spring REST API와 필터링·정렬 로직을 설계하고 사용자 역할별 데이터 조회·표현 구조 구성
- 검색 시마다 API를 호출하던 구조를 화면 진입 시 비동기 선조회 후 Recoil에 저장하는 방식으로 변경
- React Native 기반 iOS·Android 공통 기능, Firebase FCM 실시간 알림, 플랫폼별 배포·운영 이슈 대응

**[성과]**

- 대용량 데이터 대시보드 렌더링 시간을 **2.5초에서 1초대로 개선**
- React Native 앱 검색 응답 시간을 **5초에서 1초로 단축**

기술 Vue 3, React Native, TypeScript, ECharts, RealGrid, Java, Spring, REST API, MyBatis, MySQL, Recoil, SQLite, Firebase FCM